

# Performance Optimisation of Web Applications using In-memory Caching and Asynchronous Job Queues

**Sidharth S Prakash**

M.Tech in Software Systems  
Department of Information Technology  
School of Engineering, CUSAT  
Kochi, India  
sidharthstephen@gmail.com

**Binsu C Kovoov**

Assistant Professor  
Department of Information Technology  
School of Engineering, CUSAT  
Kochi, India  
binsu.kovoov@gmail.com

**Abstract** -- Web applications are an integral part of the internet domain. They provide services to the clients from a remote web server and hence the quality of the service to the clients depends on its performance such as speed and response time. This paper presents two specific ways to boost the performance of a web application from two entirely different perspectives. First method utilises in-memory caching, which aims to improve the speed of database retrieval process using a cached data set. In the second method, the user perceived delay is decreased by pushing the heavy time consuming tasks to the background from the main user interface thread. The paper evaluates the performance of in-memory caching and asynchronous job queues in different scenarios using statistical analysis tools.

**Keywords** -- Web application; In-memory caching; Asynchronous job queue; Statistical analysis.

## I. INTRODUCTION

Internet is ubiquitous in this modern world and a company or governmental organisation may provide their services to its clients through the internet in form of web applications. Most people access these services through the internet with the help of a web browser. Web application is a client-server application where the client side runs on the web browser and the server side runs on a remote web server.

The performance of a web server is the most important factor behind the success of a web application [1]. It is a very broad term that encompasses the speed, utility, usability and security of an application. This paper focuses on the aspect of speed and discusses two specific techniques to achieve it. The first technique is the utilisation of in-memory caching which aims to improve the speed of database retrieval process using a cached data set [2]. The second method is the usage of asynchronous job queues which can decrease the effective user perceived delay by pushing the heavy time consuming processor intensive tasks to the background from the main user interface thread.

The database operations are one bottleneck that web applications possess in terms of speed [3]. Especially heavyweight queries with lot of SQL join operations can take considerable amount of time to process and produce results.

The reads from the database can be speeded up using the concept of caching where the heavyweight query results are cached. When a database read is requested by the client, these cached values are used rather than re-executing the queries again.

Users expect the user interface to be responsive throughout the user interaction session. When performing a heavy processor intensive task, the user interface may remain frozen till this heavy task is performed and prevent the user from interacting with the user interface. The effective user perceived delay can be reduced significantly by farming out the heavy tasks to a background task runner through a job queue. Thus the user can interact with the user interface as it doesn't freeze due to the execution of heavy processor intensive task.

The paper is organised as follows. The first section deals with in-memory caching concept and the algorithm employed for database fetch request in a cached environment. The next section deals with the concept of asynchronous job queues and how a client can farm out a job to a task runner through it. The experiment methodology section follows the former section giving brief description on the tools and methodologies applied in the tests. Analysis and observation section analyses the statistical results obtained during the experimentation procedure. The paper concludes by drawing inferences from the analysis and observation of statistical results obtained during the experiment.

## II. IN-MEMORY CACHING

Database normalisation is the process of eliminating the data redundancy through specific organisation of data in form of multiple tables in the database. Thus a database table alone cannot produce the complete details about a database entity and requires SQL join operations for this purpose. Clients usually communicate with the web server through an application programming interface (API) which produces a structured reply for every valid request.

If the basic details about a database entity are cached, the web server can respond to the requests using the values fetched directly from the cache. Caches are storage spaces

other than the hard disk which aims at high speed retrieval of data. The caches work on the principle of “locality of reference” which states that the data set once accessed, has a higher probability of access at a later stage [4]. Database caching aims at caching the results of heavy weight SQL queries, so that the web server need not re-execute the query during a later request but rather fetch the results from the cache. This process considerably increases the speed of the web server in responding to a user query [5].

#### A. In-memory Caching Algorithm

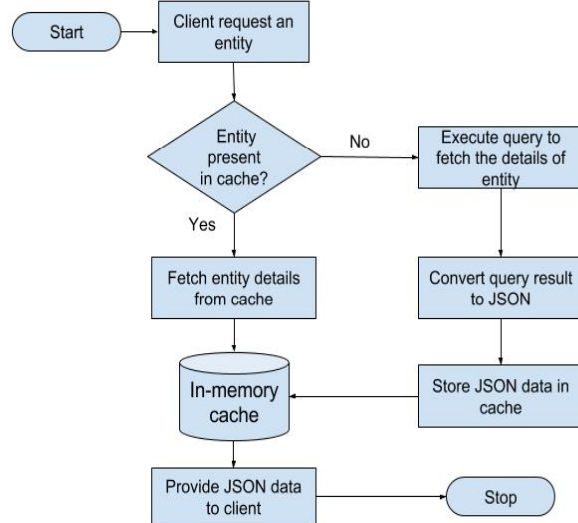


Fig. 1. Algorithm to fetch entity details in a cached environment

The database cache can be implemented using an in-memory key-value storage mechanism using the algorithm presented in Fig. 1. When a client requests for a specific data set, the web server first checks the in-memory cache store using the entity key. If the requested data set is absent, the server executes the SQL query. The query result is converted to JavaScript Object Notation (JSON) and stored as the value of the requested entity key. If the requested data is present in the cache, it is directly fetched and provided to the client. The cached data will become stale when the actual source of data is updated. Hence an asynchronous task runner is employed which updates the cache whenever the source data is updated.

#### III. ASYNCHRONOUS JOB QUEUES

The job queues are used by background task runners to process a task in asynchronous mode. In synchronous mode of processing the task submitter will wait until the task runner has completed the processing. But whereas in asynchronous mode the task submitter will never be blocked and resumes its processing once the task is submitted to the background task runner [6].

The interaction between a client and a background task runner is through a job queue as depicted in Fig. 2. The client submits a task into the job queue shared with the background task runner. The task runner checks the presence of new jobs

in the queue periodically and fetches the task from the job queue and performs the operation in the background.

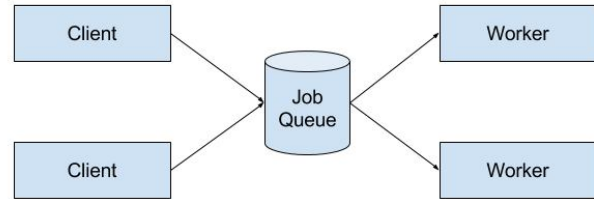


Fig. 2. Asynchronous interaction between client and worker

The greatest advantage background task runners provide is a non-blocking user interface. The time consuming heavy task is pushed into the job queue by the client for the worker to fetch and process. Utilising the asynchronous mode of processing, the client will never wait for the task to finish. This ensures that the user perceived delay will be minimised and the user interface is not frozen for a considerable amount of time. If we rather use synchronous mode of processing, the client and in turn the user will have to wait for the task to finish and will be presented with a frozen user interface during the processing time.

#### IV. EXPERIMENT METHODOLOGY

The web applications are deployed on web servers to serve each client's requests and hence to analyse the web server performance, the client request traffic load must be simulated. Apache benchmark is a tool for HTTP server benchmarking which produces test results for analysing the server performance in various scenarios [7]. Tool provides the facility to simulate web traffic with specifications like the number of requests and number of concurrent requests. The test scenarios are simulated using this benchmarking tool and the server performance in serving each request can be saved into a file for further statistical analysis.

R is a programming language for the statistical analysis of data sets [8]. The result of apache benchmark for a specific test scenario saved in a file must be analysed relative to other relevant test scenario results. Hence the benchmarking results to be compared relatively are fed into R for their statistical analysis. R can provide visual representation and easier data analysis through graphical plots.

In-memory caching is implemented using in-memory database called Redis for the experimentation [9]. There are other choices available like Memcached, but Redis is the most popular in-memory key value store for commercial application development because it can implement persistent data storage. Background task runners are implemented using Gearman which is a application framework that can be used to farm out tasks to other machines [10]. Gearman supports a client, job queue and worker architecture which can run large tasks efficiently in asynchronous mode.

The test scenarios for analysing the effect of in-memory caching and background task runners are as follows. The

performance of web application with in-memory caching and without caching are evaluated against the heaviness of the SQL query statement and the number of concurrent client requests. Relatively heavier queries are created using more number of SQL join statements so that the queries becomes more processor intensive and takes more time to complete execution. The performance of the web application with background task runners and without task runners is evaluated against the time taken to execute a task and the number of concurrent client requests. To obtain uniform network conditions, all the optimisation tests were done on uniform bandwidth.

## V. ANALYSIS AND OBSERVATIONS

This section is divided into two sections which explain the observations derived from the plots created through R using the benchmarking results for the test scenarios of in-memory caching and asynchronous job queues.

### A. In-memory Caching

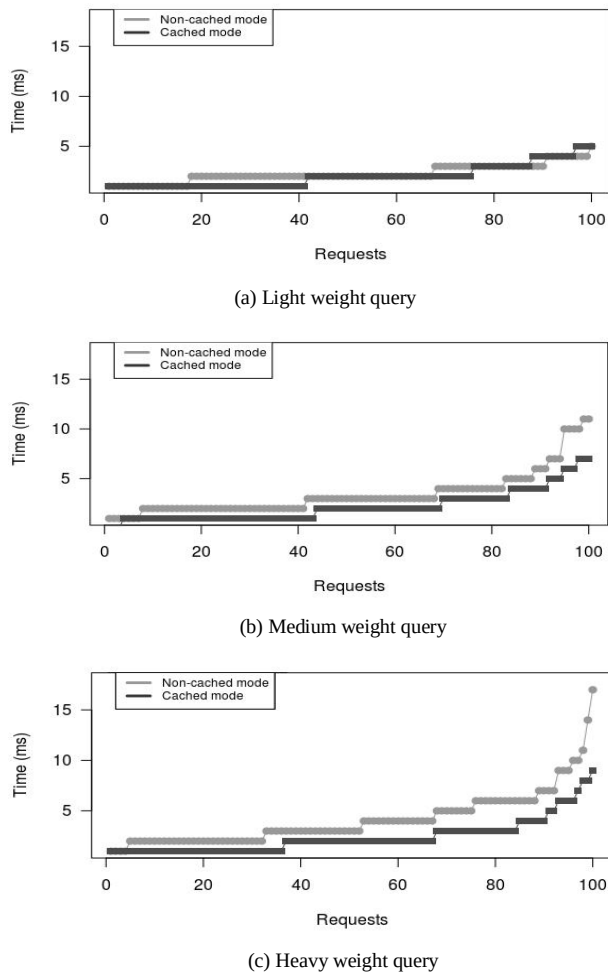


Fig. 3. Time taken for executing different weighted queries

A simple query with no join operations is chosen as a light weight query for experimentation. It is evident from Fig. 3. (a) that the cached performance and non-cached performance is

almost the same in the case of light weight query execution. The Fig. 3. (b) provides an impression that the cached performance is slightly better than the non-cached performance in case of medium weight query created with four join operations. A significant boost in performance of heavy weight query processing is noted in the Fig. 3. (c) for cached implementation than in the case of non-cached implementation. A query with eight join operations was selected as the heavy weight query for testing.

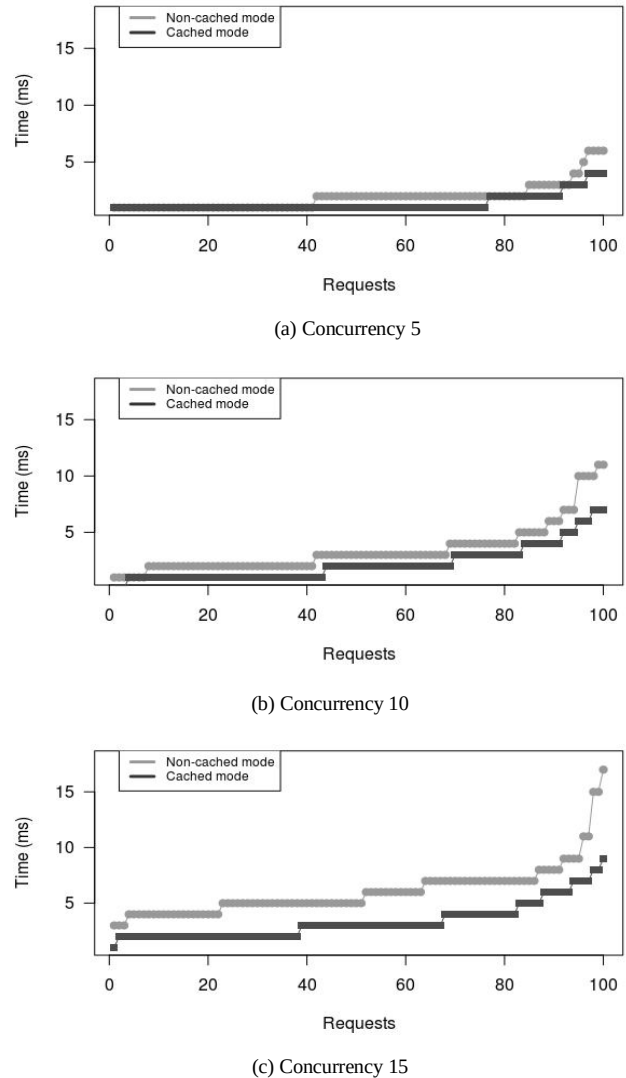
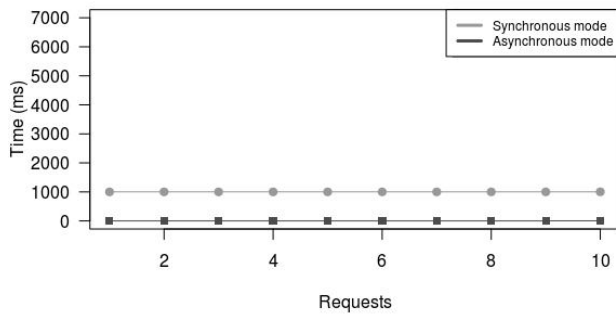


Fig. 4. Time taken for medium weighted query for different concurrency

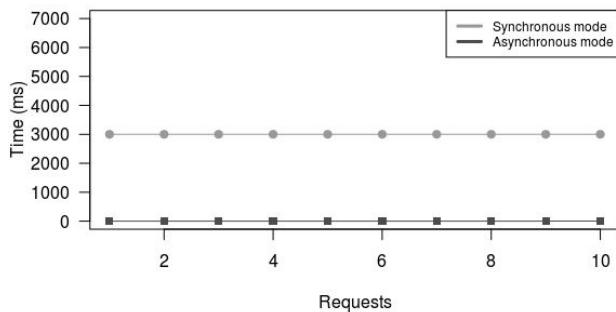
The overlap between the two experimentation plots in the beginning stage is clear from Fig. 4. (a). But there exist a slight performance improvement using the cached mode of implementation towards the end of request phase when the number of concurrent users is 5. A satisfactory performance improvement can be noted from the Fig. 4. (b) for the cached implementation than in the absence of caching when 10 users hit the server simultaneously. There is a significant boost in the performance when tested with 15 concurrent users for the

cached implementation over the non-cached implementation as observed from the Fig. 4. (c).

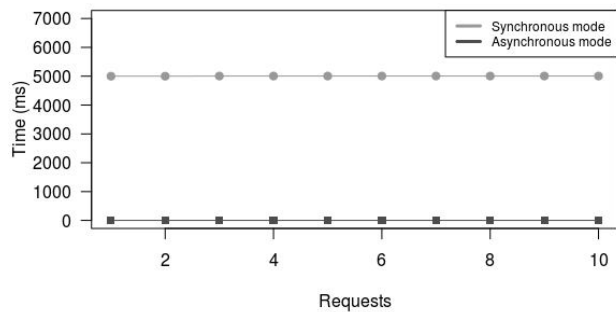
### B. Asynchronous Job Queues and Task Runners



(a) 1 second



(b) 3 seconds

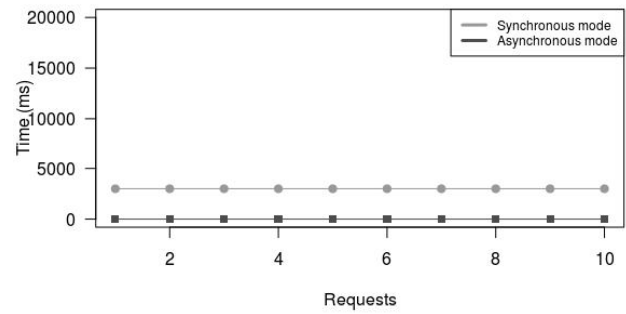


(c) 5 seconds

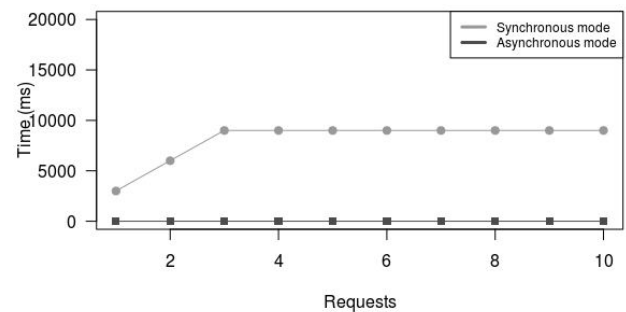
Fig. 5. User perceived delay for tasks of different lengths

The user perceived delay while executing a task synchronously is directly proportional to the length of the task. The delay experienced by user's in executing a task of length one second asynchronously through the background task runner is less than executing it directly as seen in Fig. 5. (a). It is comprehensible from Fig. 5. (b) that the delay perceived in executing the task of three seconds asynchronously, is satisfactorily less compared to synchronous execution. There is significant improvement in performance as evident from Fig. 5. (c), when executing the task of five seconds asynchronously using a background task runner. Another conclusion that could be drawn from the above plots is that the

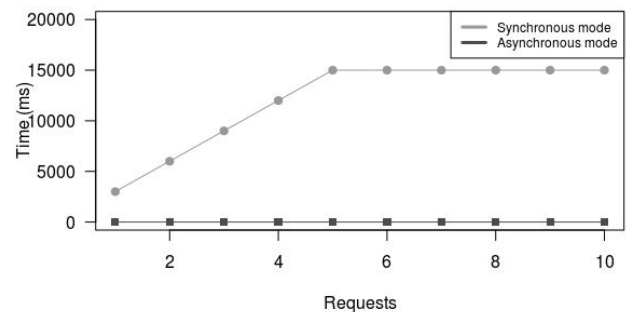
delay perceived when using asynchronous mode of processing is very close to 0 milliseconds.



(a) Concurrency 0



(b) Concurrency 3



(c) Concurrency 5

Fig. 6. User perceived delay for task of 3 seconds for different concurrency

It is obvious from Fig. 6. (a) that the delay observed by users is reduced when executing the task in asynchronous mode with no concurrent requests. Whereas in the case of three concurrent users as seen in Fig. 6. (b), the performance of the asynchronous task runners is better than the previous case of zero concurrency. The Fig. 6. (c) shows a very significant reduction in delay for executing the tasks in asynchronous mode when five users concurrently request the server. Hence it is evident that the delay experienced by users while executing a task synchronously is directly proportional to the number of concurrent requests.

## VI. CONCLUSION

Web application performance can be improved by certain design choices. But the optimisations employed will not give satisfactory results in every scenario. In the case of in-memory caching, this design choice will produce a performance boost only when there exists heavy weight queries in the application which are processor intensive and time consuming. When the results of heavy weight queries are cached, the web server can easily use this cached data set to serve the client requests. Moreover, higher the number of concurrent requests, higher is the reduction in the time taken to serve the requests. The performance of in-memory caching depends on the RAM available on the server. Additionally if data persistence is employed, the hard disk space available on the server also plays a role in the success of optimisation.

In case of background task runners and job queues, this design choice will generate performance improvement when there are long tasks that could be executed in the background asynchronously in the web application. The longer the time taken for the task, the higher the reduction in user perceived delay when using background task runners. Again as in the case of in-memory caching, as the number of concurrent users increases, the performance of the web application is improved using the background task runners. This is because as the number of concurrent users increases, effective time taken for the tasks tends to increase. Hence it is vital to analyse and understand the traffic a web application will be subjected to and other specific requirements so as to employ the best performance optimisation technique.

## REFERENCES

- [1] L. Z. Zhu and F. Li. The Collection and Analysis of Performance Parameters in Web applications. *International Conference on Electronics and Optoelectronics (ICEOE)*, 150-154, July 2011.
- [2] S. Sivasubramanian, G. Pierre, M. V. Steen, G. Alonso. Analysis of Caching and Replication Strategies for Web Applications. *IEEE Internet Computing*, 11(1): 60-66, January 2007.
- [3] A. Atserias, M. Grohe, D. Marx. Size Bounds and Query Plans for Relational Joins. *IEEE 49th Annual IEEE Symposium on Foundations of Computer Science*, 739-748, October 2008.
- [4] G. Karakostas, D. N. Serpanos. Exploitation of different types of locality for Web caches. *Seventh International Symposium on Computers and Communications*, 207-212, 2002.
- [5] J. Yan, J. Chen, W. Jiang. Data Caching Techniques in Web Application. *Enterprise Systems Conference*, 289-293, August 2014
- [6] D. Dig. Refactoring for Asynchronous Execution on Mobile Devices. *IEEE Software*, 32(6): 52-61, December 2015.
- [7] M. Olson, K. Christensen, S. Lee, J. Yun. Hybrid web server: Traffic analysis and prototype. *IEEE 36th Conference on Local Computer Networks (LCN)*, 131-134, October 2011.
- [8] S. Voulgaropoulou, G. Spanos, L. Angelis. Analyzing Measurements of the R Statistical Open Source Software. *35th Annual IEEE Software Engineering Workshop (SEW)*, 1-10, October 2012.
- [9] D. Stjepanovic, M. Savic, J. Jokić, S. Marić. Performance measurements of some aspects of multi-threaded access to key-value stores. *23rd Telecommunications Forum Telfor (TELFOR)*, 831-834, November 2015.
- [10] K. T. Hsu, S. Sinha, Y. C. Pi, T. Y. Ho. A Hierarchy-Based Distributed Algorithm for Layout Geometry Operations. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 31(10): 1546-1557, September 2012.